



随机梯度下降法

- 基于最大似然估计的误差函数由多个项组成，每个数据点对应一项，适用于一组独立观测值

$$E(p;w,q) = \sum_{n=1}^N E_n(p;w,q).$$

- 针对大规模数据集最广泛使用的训练算法基于梯度下降的序列版本，即随机梯度下降或在线梯度下降。
- 该算法每次仅基于单个数据点更新权重向量

$$w^{(p+1)} = w^{(p)} - \eta \nabla E_n(w^{(p)})$$



- 完整遍历整个训练集称为一个训练 **epoch**。



- SGD处理数据冗余的效率更高。
- 取一个数据集，通过复制每个数据点使其规模翻倍。
- 这仅使误差函数乘以2倍，因此相当于使用原始误差函数。
- 批处理方法需要双倍计算量来评估梯度，而 SGD 则不受影响。
- SGD 能够逃离局部最优解，因为对于整个数据集而言，误差函数的驻点通常不会是每个数据点的驻点。



- 基于单个数据点计算的梯度，对完整数据集梯度的估计存在显著噪声。
- 折衷方案（批处理与随机）采用称为小批量的数据子集，在每次迭代中评估梯度。

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_{n:n} \quad \mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_{n:n} \quad \mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_{n:n}$$



小批量的最优规模

- 从 N 个样本计算均值的误差由 σ^2/N 给出：递减
通过增大批量大小来提高真实梯度估计的准确性。
- 高效利用代码运行所在的硬件架构

序列相关性

- 构成数据点应从数据集中随机选取



- 具体的初始化方式会显著影响
 - 达到解所需的时间
 - 以及所训练网络在泛化性能上的表现

对称性破缺

用于初始化权重的分布通常为

- 要么是区间 $[-\epsilon, \epsilon]$ 内的均匀分布
- 或零均值高斯分布，其形式为 $N(0, \epsilon^2 \mathbf{Q})$ 。

误差反向传播



- 本节目标是为前馈神经网络的误差函数 E_{pwq} 求解梯度，寻找高效评估技术。
 -
- 这可通过局部消息传递方案实现，该方案中信息在网络中交替向前和向后传递
- 该方法被称为误差反向传播，有时简称为反向传播。



误差反向传播：训练过程的本原

- 训练包含一系列调整权重的步骤，每个步骤分为两个阶段：
 1. 计算误差函数对权重的导数。
 - 反向传播在提供计算高效的导数评估方法方面具有重要意义。
 - 误差通过网络向后传播。
 2. 随后使用导数计算所需调整值。
 - 最简单的此类技术涉及梯度下降法。



误差反向传播：训练过程的本质

- 这两个阶段是截然不同的。
- 第一阶段即通过网络向后传播误差以计算导数，该方法不仅适用于多层感知器，还可推广至其他多种网络结构。
- 该方法同样适用于除简单平方和之外的其他误差函数，以及雅可比矩阵和赫塞矩阵等其他导数的计算。
- 第二阶段利用计算出的导数调整权重，可采用多种优化方案实现，其中许多方案的优化能力远超简单的梯度下降法。



误差函数导数的评估

- 对于一组独立同分布数据，由最大似然法定义的误差函数包含多个项，每个训练集数据点对应一项

$$E_{\mathbf{p}|\mathbf{w}|\mathbf{q}} = \sum_{n=1}^N E_n[\mathbf{p}|\mathbf{w}|\mathbf{q}]$$

- 一种线性模型，其中输出 y_k 是输入变量 x_i 的线性组合

$$y_k = \sum_i w_{ki} x_i$$

- 同时包含一个误差函数，对于特定输入模式 n ,

$$E_n = \frac{1}{2} \sum_k (y_{nk} - t_{nk})^2$$



误差函数导数的评估

- 考虑单个数据点的误差函数，其关于权重 w_{ji} 的梯度由下式给出

$$\frac{\partial E_n}{\partial w_{ji}} = p_{nj} - t_{nj} q_{x_{ni}}$$

- 可解释为涉及以下乘积的“局部”计算：
 - 与链路输出端关联的“误差信号” $y_{nj} - t_{nj}$
 - 变量 $x_{(ni)}$ (与链路输入端相关联)。



- 在一般的前馈网络中，每个单元对其输入进行加权求和，形式为：

$$a_j = \sum_i w_{ji} z_i \quad (\text{加权和})$$

- 其中 z_i 是向单元 j 发送连接的单元或输入的激活值，而 w_{ji} 是该连接对应的权重系数。
 j ， w_{ji} 是该连接对应的权重。

- 该和式通过非线性激活函数 h 进行转换 p_q 从而得到单元 j 的激活值 z_j ，其形式为

$$z_j = h(p_q a_j) \quad (\text{激活值})$$



- 对于训练集中的每个模式，我们假设
 - 我们已向网络提供对应输入向量
 - 并通过连续应用上述两个方程计算了网络中所有隐藏层和输出层单元的激活值。
- 该过程常被称为前向传播
 - 因为它可视为信息在网络中的前向流动。



- 让我们考虑对 E_n 关于权重 w_{ji} 的导数进行评估。
- 各单元的输出将取决于输入模式 n 。
- 但为保持符号简洁，我们省略下标 n
 -
- 首先注意到， E_n 仅通过单位 j 的总和输入权重 w_{ji} 影响 $w_{(ji)}$ 。
 a_j 之和。



- 因此我们可以应用偏导数的链式法则得到

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{\partial E_n}{\partial a_j} \frac{\partial a_j}{\partial w_{ji}} \quad (\text{链})$$

- 我们引入一种有用的错误标记法

$$\delta_j = -\frac{\partial E_n}{\partial a_j}$$

- 利用（加权和），我们可以写成

$$\frac{\partial a_j}{\partial w_{ji}} = z_i$$



- 将上述两个方程代入（链式），我们得到

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i \quad (\text{链式*})$$

- 方程（链式法则）告诉我们，所需的导数只需通过将权重输出端单元的 δ 值与权重输入端单元的 z 值相乘即可获得（其中 z 取值为1）。
- 请注意，这与本节开头讨论的简单线性模型具有相同形式。因此，要计算导数，只需分别计算网络中每个隐藏层和输出单元计算其 δ 值，再应用(链式法则*)即可。



- 对于输出单元，我们有

$$\delta_k = y_k - t_k \quad (\text{误差})$$

- 只要我们使用规范链接作为输出单元的激活函数。
- 为评估隐藏单元的 δ 值，我们再次运用偏导数的链式法则：

$$\delta_j = \sum_k \delta_k \frac{\partial E_n}{\partial a_j} \frac{\partial a_k}{\partial a_j}$$

其中求和遍及所有接收单元 j 连接的单元 k 。

- 标记为 k 的单元可能包含其他隐藏单元和/或输出单元。



- 若将 δ 的定义代入前式，并结合式(加权求和)与式(链式关系)，可得出以下反向传播公式：

$$\delta_j = h_j' \sum_k w_{kj} \delta_k \quad (\text{反向传播})$$

- 这表明特定隐藏单元的 δ 值可通过从网络上层单元向下传播 δ 值获得。
- （反向传播）中的求和操作是对 w_{kj} 的第一个索引进行的（对应于信息在网络中的反向传播）。



- 而在前向传播方程（forward）中，它是通过第二个下标进行求和。
- 由于我们已知输出单元的 δ 值，因此通过递归应用式（反向传播），即可计算出前馈网络中所有隐藏单元的 δ 值，无论其拓扑结构如何。



- 因此，误差反向传播过程可概括如下：
 1. 将输入向量 x_n 输入网络，通过（加权求和）和（激活函数）进行前向传播，求得所有隐藏层和输出层单元的激活值。
 2. 利用公式（误差）计算所有输出单元的 δ_k 。
 3. 通过（反向传播）将 δ 值反向传播，以获得网络中每个隐藏单元的 δ_j 。
 4. 使用 (chain*) 计算所需的导数。



- 对于批处理方法，可通过对训练集中的每个模式重复上述步骤，然后对所有模式求和，从而获得总误差 E 的导数：

$$\frac{\partial E}{\partial w_{ji}} = \sum_n \frac{\partial E_n}{\partial w_{ji}}$$

- 在上述推导中，我们隐含地假设网络中的每个隐藏单元或输出单元都具有相同的激活函数 $h_p q$ 。
- 然而该推导可轻易推广，允许不同单元采用各自独立的激活函数，只需追踪 $h_p q$ 的具体形式即可。
与哪个单元相关联。

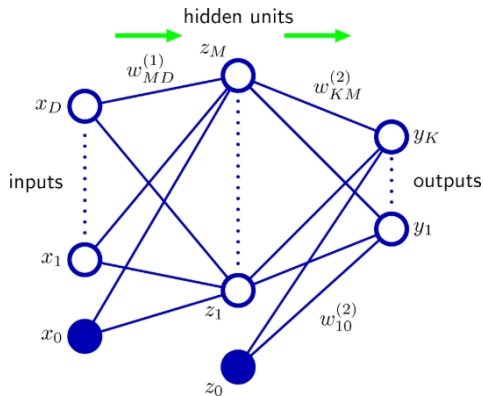


误差反向传播：一个示例

- 上述反向传播过程的推导允许误差函数、激活函数和网络拓扑结构采用通用形式。
 -
- 为深入理解该算法，我们通过具体实例进行阐释。
- 该示例兼具简单性和实用价值。



误差反向传播：一个示例



- 如右图所示，这是一个两层（三层）网络

- 其输出单元采用线性激活函数， $y_k a_k$
- 其隐藏层单元采用双曲正切
激活函数，由下式给出

$$\tanh$$

$$\frac{e^{2x} - 1}{e^{2x} + 1}$$

- 采用标准均方误差函数，因此

$$E_n = \frac{1}{2} \sum_{k=1}^K y_k^2$$



$t_k q$

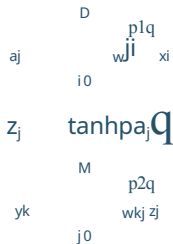


误差反向传播：一个示例

- 该函数的一个有用特性是其导数可表示为：

$$h^1 \text{ paq } 1 \text{ hpaq}^2 .$$

- 对于训练集中的每个模式，我们依次先执行前向传播：





误差反向传播：一个示例

- 接下来，我们使用以下公式计算每个输出单元的 δ 值

$$\delta_k = y_k - t_k$$

- 然后通过反向传播获得隐藏单元的 δ 值，使用

$$\delta_j = \sum_k w_{kj} \delta_k$$

- 最后，关于第一层和第二层权重的导数由以下公式给出

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j x_i$$

$$\frac{\partial E_n}{\partial w_{kj}} = \delta_k z_j$$



- 反向传播最重要的方面之一是其计算效率。
- 要理解这一点，让我们考察评估误差函数导数所需的计算机运算次数如何随网络中权重和偏置的总数 w 而变化。
- 当权重数 w 足够大时，单次误差函数评估（针对特定输入模式）需要 $O(w)$ 次运算。



- 除极稀疏连接的网络外，权重数量通常远大于神经元数量。
 - 因此，前向传播中大部分计算工作量都集中在求解式(1)中的求和项（加权求和），
 - 而激活函数的计算仅占很小的开销。
- （加权求和）中的每项都需要一次乘法和一次加法运算，导致整体计算成本为 $O_p W q$ 。



- 计算误差函数导数的另一种替代方法是采用反向传播中的有限差分法。
- 具体方法是依次对每个权重施加 ϵ ! 1, 并
用表达式

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{E_n(p_{w_{ji}+\epsilon}) - E_n(p_{w_{ji}-\epsilon})}{2\epsilon} \quad \text{Op} \epsilon q \quad (\text{牛顿差商})$$

- 在软件模拟中，通过缩小 ϵ 值可提高导数近似值的精度，直至出现数值舍入误差问题。



- 采用对称中心差分法可显著提高有限差分法的精度，其形式为

$$\frac{\frac{B E_n}{B w_j} - \frac{E_n - p w_{ji}}{2 \varepsilon} \frac{\varepsilon q}{E_n - p w_{ji}} - \frac{E_n - p w_{ji}}{2 \varepsilon} \frac{\varepsilon q}{E_n - p w_{ji}}}{2 \varepsilon} \approx O p \varepsilon^2 q$$

- 在此情况下， $O p \varepsilon q$ 项的修正量相互抵消，这可通过泰勒展开验证，因此残余修正量为 $O p \varepsilon^2 q$ 。
- 然而，计算步骤数约为（牛顿差商法）的两倍。



- 数值微分的主要问题在于其高度理想的 $O_p W q$ 缩放特性已丧失。
 - 每次正向传播需要 $O_p W q$ 步，
 - 而网络中存在 W 个权重，每个权重都必须单独扰动，因此整体标度为 $O_p W^2 q$ 。



- 然而，数值微分在实践中起着重要作用，
- 因为将反向传播计算的导数与中心差分法获得的导数进行比较，可为任何反向传播算法的软件实现提供强有力的正确性检验（梯度检查）。

在实际训练神经网络时，应采用反向传播计算导数，因其能提供最高精度与数值效率。

神经网络中的正则化



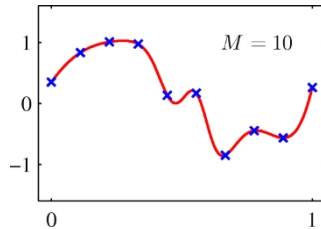
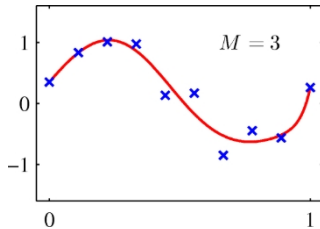
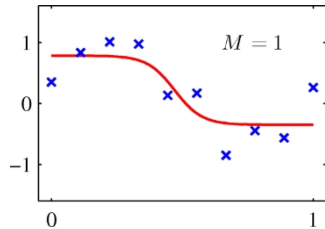
- 神经网络中输入与输出单元的数量通常由数据集的维度决定，
- 而隐藏层单元数 M 则为自由参数，可通过调整获得最佳预测性能。
- 需注意 M 控制着网络参数（权重与偏置）的数量，
- 因此在最大似然估计框架下，可预期存在使泛化性能最优的 M 值，
- 即在欠拟合与过拟合之间达到最优平衡。



- 基于正弦数据集抽取的10个数据点训练的双层网络示例。
- 图表展示了通过缩放共轭梯度算法最小化均方误差函数，分别对具有M 1、3和10个隐藏单元的网络拟合结果，通过缩放共轭梯度算法最小化平方和误差函数获得。

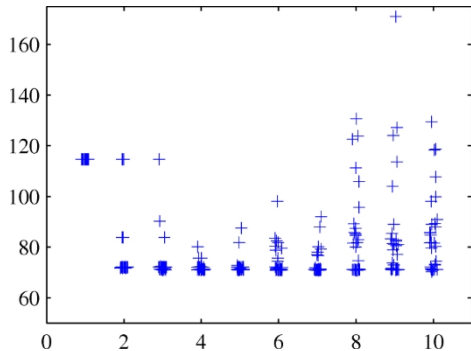


- 正弦回归问题中不同 M 值效果的示例。





- 然而，由于误差函数中存在局部极小值，泛化误差并非 M 的简单函数。
- 让我们观察在不同 M 值范围内，为权重向量选择多个随机初始化的效果——整体最佳验证集性能出现在 $M=8$ 时。



- 多项式数据集的测试集平方和误差与网络隐藏单元数量的关系图，每个网络规模采用30次随机初始化，展示局部极小值的影响。
- 每次新初始化时，权重向量均通过采样自均值为零、方差为10的各向同性高斯分布进行初始化。



神经网络中的正则1

- 另一种方法是选择一个相对较大的M值，然后通过误差函数中添加正则化项来控制复杂度。

- 最简单的正则化项是二次项，其对应的正则化误差为：

$$E_{\text{reg}} = \frac{\lambda}{2} \mathbf{w}^T \mathbf{w}$$

- 该正则化项亦称为权重衰减。
- 有效模型复杂度由正则化系数 λ 决定。
- 该正则化项可解释为权重向量 $\mathbf{w}$ 上零均值高斯先验分布的负对数。



一致的高斯先验分布

- 简单权重衰减的一个局限性在于，它与网络映射的某些缩放特性不相容。
- 考虑一个将 t_{x_i} $\mathbf{u}$ 映射到 t_{y_k} $\mathbf{u}$ 的多层感知机。
- 第一隐藏层中隐藏单元的激活值形式为：

$$z_j = \sum_i h_{ji} w_{ji} x_i + w_{j0}$$

- 而输出单元的激活函数由以下公式给出：

$$y_k = \sum_j w_{kj} z_j + w_{k0}$$



一致的高斯先验分布

- 假设我们对输入数据执行如下形式的线性变换：

$$\tilde{x}_i = ax_i + b.$$

- 那么，通过对输入到隐藏层单元的权重和偏置进行对应的线性变换，我们可以使网络执行的映射保持不变，其形式为：

$$\begin{aligned} \tilde{w}_{ji} &= \frac{1}{a} w_{ji} \\ \tilde{w}_{j0} &= \frac{1}{a} w_{j0} - \frac{b}{a} \end{aligned}$$



- 同样地，网络输出变量的线性变换形式为

$$y_k = \tilde{y}_k + cy_k + d$$

- 可通过对第二层权重和偏置进行如下变换实现：

$$\begin{aligned} w_{kj} &= \tilde{w}_{kj} + cw_{kj} \\ w_{k0} &= \tilde{w}_{k0} + cw_{k0} + d \end{aligned}$$



- 若我们使用原始数据训练一个网络，同时使用经上述线性变换处理输入和/或目标变量的数据训练另一个网络，
- 那么一致性要求我们应得到等效的网络，它们仅在给定的权重线性变换上存在差异。
- 任何正则化器都应符合此特性，否则它将任意偏袒某一等效解方案。
- 显然，简单权重衰减（将所有权重和偏置项等同对待）并不满足此特性。



- 寻找在四种线性变换下不变的正则化器
- 正则化器应保持对权重重新缩放和偏置位移的不变性。
- 此类正则化项可由下式给出

$$\frac{\lambda_1}{2} W_1^2 + \frac{\lambda_2}{2} W_2^2 \quad (\text{非正规化项})$$

- 其中 W_1 表示第一层的权重集合， W_2 表示第二层的权重集合，且不包含偏置项。
- 该正则化项在权重变换下保持不变，其中参数通过 $\lambda_1 \rightarrow a_1^{1/2} \lambda_1$ 和 $\lambda_2 \rightarrow a_2^{1/2} \lambda_2$ 进行重新缩放。



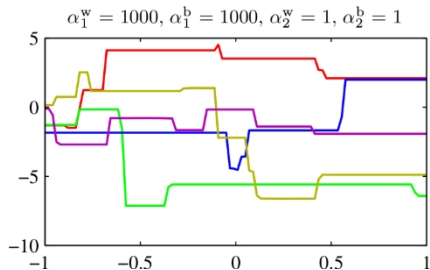
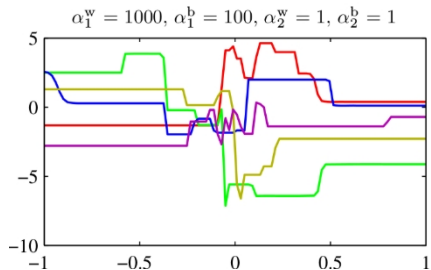
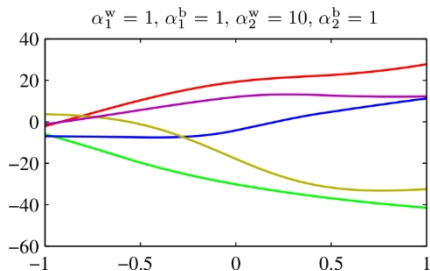
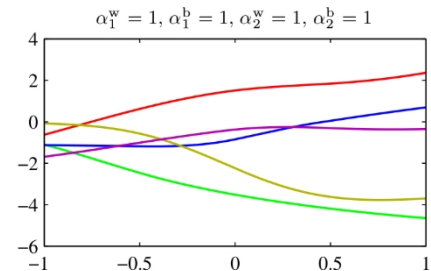
- 该（非正规化正则项）对应于形式为

$$p(w|\alpha) \propto \exp \left[-\frac{\alpha_1}{2} w^2 - \frac{\alpha_2}{2} w^2 \right]$$

- 这种形式的先验是不恰当的（无法归一化），因为偏置参数不受约束。
- 在贝叶斯框架内，使用非正规先验会导致正则化系数选择困难及模型比较困难，因为对应的证据为零。
- 因此通常会为偏置项设置独立先验（这将破坏位移不变性），并赋予其专属超参数。



一致的高斯先验分布





- 我们可通过从先验分布中抽样并绘制对应网络函数，来展示最终形成的四个超参数的影响。
- 图示说明超参数对权重与偏置先验分布的影响：该两层网络包含单一输入、单一线性输出，以及12个采用"tanh"激活函数的隐藏单元。
- 先验分布由四个超参数 α^b_1 、 α^w_1 、 α^b_2 和 α^w_2 控制，其中
分别表示第一层偏置项、第一层权重、第二层偏置项及第二层权重的高斯分布精度。



- α^w 控制函数的垂直尺度（注意上方两个图中垂直轴范围的不同），
- α^w 控制函数值变化的水平尺度，
- 而 α^b 控制变化发生的水平范围。
- 参数 α^b （此处未示出）控制垂直偏移的范围的范围。



- 更普遍地，我们可以考虑将权重划分为任意数量组 W_k 的先验分布，使得

$$p(\mathbf{w}) \propto \exp \left\{ -\frac{1}{2} \sum_k \alpha_k \sum_{j \in W_k} w_j^2 \right\}$$

- 其中

$$\sum_{j \in W_k} w_j^2 = \sum_{j \in W_k} w_j^2.$$

- 作为此先验的特殊情况，若将组对应于每个输入单元关联的权重集合，并针对对应参数 α_k 优化边缘似然，则可得到自动相关性判定。



- 作为控制网络有效复杂度的一种替代正则化方法，可采用提前终止程序。
 -
- 非线性网络模型的训练过程对应于基于训练数据集定义的误差函数的迭代缩减。
- 对于许多用于网络训练的优化算法（如共轭梯度法），误差是迭代指数的非递增函数。

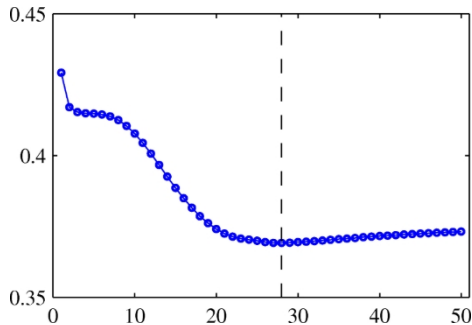
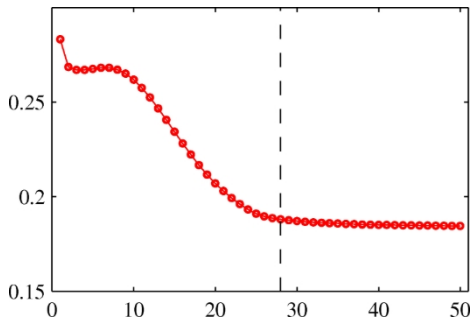


- 然而，相对于独立数据（通常称为验证集）测得的误差，通常会先下降，随后随着网络开始过拟合而上升。
- 训练可在验证数据集误差最小点终止
- 以获得具有良好泛化性能的网络。



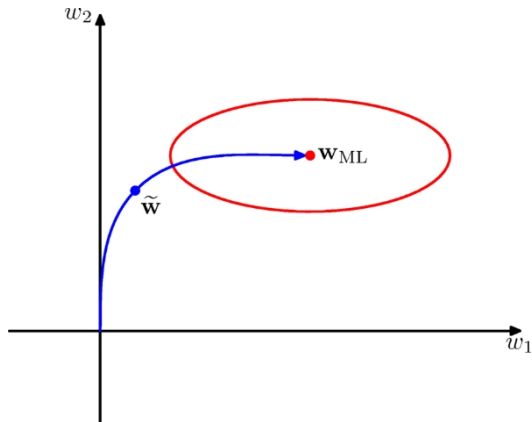
提前终止：训练误差与验证误差

- 实现最佳泛化性能的目标表明，训练应在垂直虚线所示点停止，该点对应于验证集误差的最小值。





- 这种情况下网络的行为有时可通过网络有效自由度数量进行定性解释：
- 该数值在训练初期较小，随后逐渐增大，
- 这反映出模型有效复杂度的持续提升。
- 在训练误差达到最小值之前终止训练，实质上是通过限制网络有效复杂度来实现优化。



- 对于二次误差函数，我们可以验证这一见解，并证明提前终止应表现出与使用简单权重衰减项进行正则化相似的行为。
 -
- 通过提前终止训练，可获得与简单权重衰减正则化项



- 在模式识别的许多应用中，已知预测结果应在输入变量的一项或多项变换下保持不变或不变量。
- 例如在二维图像物体分类中（如手写数字），
- 特定对象无论在图像中的位置（平移不变性）或尺寸（尺度不变性）如何变化，都应被分配相同的分类结果。



- 此类变换会显著改变原始数据——具体表现为图像像素亮度的变化——但分类系统仍应输出相同结果。
- 同样在语音识别中，沿时间轴进行的小幅非线性变形——只要保持时间顺序不变——不应改变信号的解释。



- 若训练样本数量足够庞大，神经网络等自适应模型便能学习到这种不变性，至少能近似实现。
- 这要求训练集中包含足够数量的各类变换效应示例。
- 因此，对于图像中的平移不变性，训练集应包含位于多种不同位置的物体示例。



- 然而，若训练样本数量有限，或存在多个不变性特征时，此方法可能难以实施
- 因为变换组合的数量会随变换数量呈指数级增长。
- 因此我们寻求替代方案，促使自适应模型展现所需的不变性。



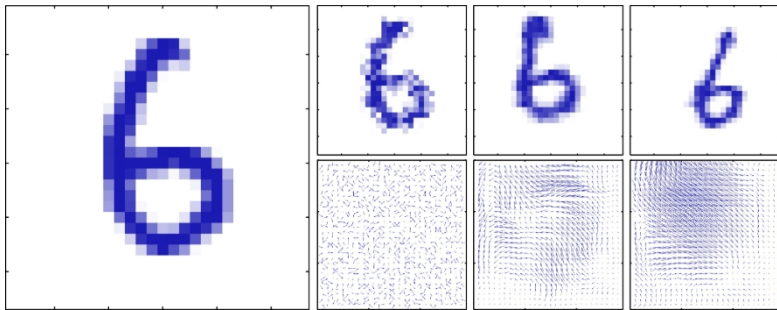
这些方法大致可分为四类：

1. 训练集通过训练样本的复本进行扩展，这些复本根据所需的不变性进行变换。
例如，在我们的数字识别示例中，我们可以为每个样本制作多个副本，在每个图像中将数字移至不同位置。
2. 在误差函数中添加正则化项，该项会对输入变换时模型输出的变化施加惩罚。由此衍生出切线传播技术。



3. 通过提取在所需变换下保持不变的特征，将不变性内置于预处理环节。任何后续采用此类特征作为输入的回归或分类系统，必然也会遵循这些不变性。
4. 最后一种方案是将不变性属性嵌入神经网络的结构中（或在相关向量机等技术中嵌入核函数的定义中）。实现这一目标的一种方法是采用局部感受野和共享权重，正如卷积神经网络中讨论的那样。

- 方法1通常相对容易实现，可用于促进复杂不变性的形成。



- 方法2保持数据集不变，但通过添加正则化项修改误差函数。



- 方法 3 的一个优势在于，它能够正确地推断出远超训练集中包含的转换范围之外的情况。
 - 然而，要找到既具备所需不变性又不会丢弃有用辨别信息的手工特征可能很困难。
- 对于序列训练算法，可通过在输入模式提交给模型前对其进行转换来实现：当模式被循环使用时，每次添加不同的转换（从适当的分布中抽取）。

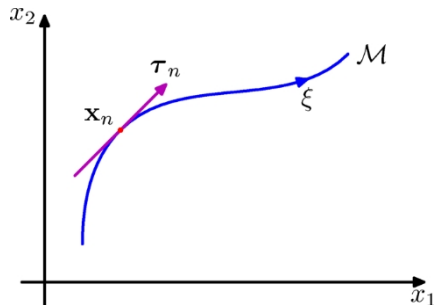


- 对于批处理方法，可通过将每个数据点复制若干次并独立转换每份副本来实现类似效果。

使用此类增强数据可显著提升泛化能力（Simard et al., 2003），但计算成本较高。



- 我们可通过切线传播技术运用正则化，促使模型对输入变换保持不变。
 -
- 考虑变换对特定输入向量 x_n 的影响。
- 只要变换是连续的（例如平移或旋转，但不包括镜像反射），
- 那么，该变换后的模式将在D维输入空间内扫出一个流形M。



- 二维输入空间示意图，展示连续变换的效果
- 当一维变换（由连续变量 ξ 参数化）作用于 x_n 时，将扫出一个一维流形 M 。
- 局部而言，该变换的效果可近似表示为切向量 τ_n 。



- 为简化起见，考虑 $D = 2$ 的情况。
- 假设变换由单一参数 ξ 控制（例如旋转角度）。
- 此时 x_n 扫出的子空间 M 为一维，并由 ξ 参数化。
- 令该变换作用于 x_n 所得的向量记为 $\text{sp}x_n, \xi q$,
- 其定义为： $\text{sp}x, 0q_x$ 。



- 则曲线 M 的切线由方向导数 $\tau_{Bs\{B\xi\}}$ 给出，而点 $x(n)$ 处的切向量由 $\tau_{Bs\{B\xi\}}$ 且点 x_n 处的切向量由

$$\tau_n = \frac{B_{sp} x_n}{B_{\xi_0}^T q_{\xi_0}}$$

- 在输入向量变换下，网络输出向量通常会发生变化。
- 输出 k 对 ξ 的导数由下式给出

$$\frac{\partial y_k}{\partial \xi_{\xi_0}} = \sum_{i=1}^D \frac{\partial y_k}{\partial Bx_i} \frac{\partial Bx_i}{\partial \xi_{\xi_0}} = J_{k,i}$$

- 其中 J_{ki} 是雅可比矩阵 J 的第 p_k 行第 iq 列元素。



- 该结果可用于修改标准误差函数，通过在原始误差函数 E 中添加正则化函数 Ω ，从而在数据点邻域内促进局部不变性，最终得到形式为

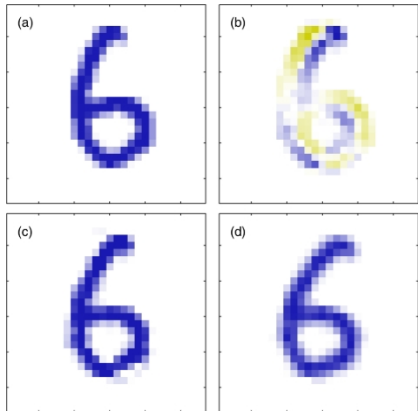
$$\tilde{E} = E + \lambda \Omega$$

- 其中 λ 为正则化系数，

$$\Omega = \frac{1}{2} \sum_{n,k} \frac{\| \mathbf{y}_{nk} - \mathbf{B} \boldsymbol{\xi}_{\xi_0} \|^2}{\sum_{n,k} \frac{1}{2} \sum_{i=1}^D \tau_{nki}^2}$$



- 当网络映射函数在每个模式向量邻域内对变换不变时，正则化函数将为零，
- 参数 λ 的取值决定了拟合训练数据与学习不变性特征之间的平衡。
- 切线向量 τ_n 可通过有限差分近似计算：
- 具体方法是：用 ξ 的微小值对原始向量 x_n 进行变换，再减去变换后的对应向量，最后除以 ξ 。



- (a) 手写数字的原始图像 x ,
- (b) 对应于顺时针无限小旋转的切线向量 τ ,
- (c) 将切向量的小量贡献添加至原始图像所得结果
果为 $x + \epsilon \tau$ (其中 $\epsilon = 15^\circ$,
- (d) 真实图像旋转后用于比较。



- 正则化依赖于通过雅可比矩阵传递的网络权重。
- 通过反向传播形式计算正则化项对网络权重的导数
可通过反向传播的扩展轻松获得。
- 若变换由 L 个参数控制
 - $L = 3$ 用于结合平面旋转的二维图像平移，
- 则流形 M 的维数为 L ，对应的正则化项由类似 Ω 的项之和构成，每种变换对应一项。
 -



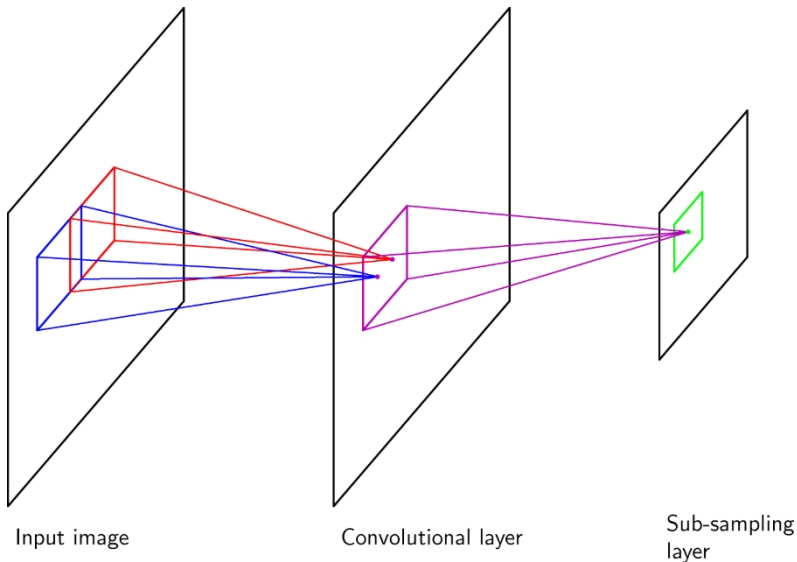
- 若同时考虑多种变换，且网络映射对每种变换均保持不变，则其将对变换组合具有（局部）不变性。
- 相关技术"切线距离"可用于在基于距离的方法中构建不变性特性。



- 促进模型对一组变换保持不变性的方法之一，是通过原始输入模式的变换版本扩展训练集。



- 另一种构建对特定输入变换具有不变性的模型的方法，是将不变性特性融入神经网络的结构中。
- 这是卷积神经网络的基础（Le Cun et al., 1989; LeCun et al., 1998），该网络已被广泛应用于图像数据处理。





- 示意图展示卷积神经网络的部分结构，包含一层卷积单元与后续的子采样单元层。
 -
- 可使用若干连续的此类层对。

反向传播的推广



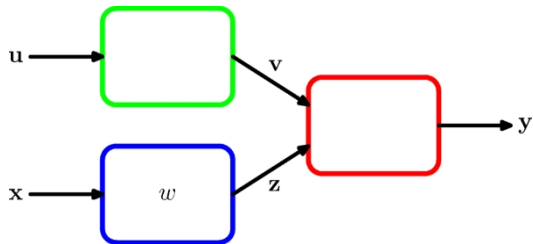
- 反向传播技术也可应用于其他导数的计算。
- 这里我们考虑雅可比矩阵的评估，其元素由网络输出相对于输入的导数给出

$$J_{ki} = \frac{\partial y_k}{\partial x_i}$$

- 其中每个此类导数都是在所有其他输入保持不变的情况下计算的。



雅可比矩阵



- 雅可比矩阵在由多个独立模块构成的系统中发挥着重要作用。
- 每个模块可包含固定或自适应函数，该函数可以是线性或非线性的，只要它可微分即可。

- 假设我们希望最小化误差函数E关于参数w的值。
- 误差函数的导数由下列公式给出：

$$\frac{\partial E}{\partial w_{k,j}} = \frac{\partial E}{\partial y_k} \frac{\partial y_k}{\partial z_j} \frac{\partial z_j}{\partial w_{k,j}}$$



雅可比矩阵

- 由于雅可比矩阵提供了输出对每个输入变量变化的局部敏感度度量
- 它还允许将任何已知的输入相关误差 Δx_i 通过训练网络进行传播，从而通过以下关系估计其对输出误差的贡献 Δy_k :

$$\Delta y_k \approx \sum_i \frac{\partial y_k}{\partial x_i} \Delta x_i$$

- 该关系在 $|\Delta x_i|$ 较小时成立。
- 一般而言，网络映射是非线性的，因此雅可比矩阵的元素并非常数，而是取决于所使用的特定输入向量而变化。



雅可比矩阵

- 雅可比矩阵可通过反向传播过程进行求解。
- 让我们首先将元素 J_{ki} 表示为以下形式：

$$J_{ki} = \frac{\partial y_{(k)}}{\partial x_i} = \sum_j \frac{\partial y_k}{\partial x_j} \frac{\partial x_j}{\partial x_i} = \sum_j w_{ji} \frac{\partial y_k}{\partial x_j}$$

- 求和遍历所有接收输入单元i连接的单元j。
- 例如，遍及先前讨论的分层拓扑结构中第一隐藏层的所有单元。



- 现在我们写出递归反向传播公式来确定导数 $\frac{\partial y_k}{\partial a_j}$

$$\frac{\partial y_k}{\partial a_j} = \sum_l \frac{\partial y_k}{\partial a_l} \frac{\partial a_l}{\partial a_j} = \sum_l h_l^1 p_{aj} q_l w_{lj} \frac{\partial y_k}{\partial a_l}$$

- 其中求和遍历所有接收单元j连接的单元l。
- 这种反向传播从输出单元开始，其所需的导数可直接从输出单元激活函数的函数形式中获得。



- 例如，若每个输出单元采用独立的s形激活函数，则

$$\frac{\partial y_k}{\partial a_j} = \delta_{kj} \sigma'(p_{aj})$$

- 而对于softmax输出，我们有

$$\frac{\partial y_k}{\partial a_j} = \delta_{kj} y_k - y_k y_j$$



我们可以将雅可比矩阵的计算过程总结如下：

- 将输入空间中待求雅可比矩阵点的对应输入向量应用于该点
- 按常规方式进行前向传播，获取网络中所有隐藏层单元和输出层单元的激活值。
 -
- 接着，针对雅可比矩阵中对应输出单元 k 的每一行 k ，使用递归关系对网络中所有隐藏单元进行反向传播。
- 最后，将反向传播推导至输入层。



- 我们已展示如何运用反向传播技术求得误差函数对网络权重的第一阶导数。
- 反向传播同样可用于计算误差的二阶导数，其表达式为：

$$\frac{\partial^2 E}{\partial w_{ji} \partial w_{lk}}$$

- 将所有权重和偏置参数视为单一向量 w 的所有元素 w_i ，记为 w ，这样考虑较为方便。
- 二阶导数构成海森矩阵 H 的元素 H_{ij} ，其中 $i, j = 1, \dots, W$ 且 W 为权重与偏置的总数。



- 海森矩阵在神经计算的诸多方面发挥着重要作用，包括：
 1. 若干用于训练神经网络的非线性优化算法，基于对误差表面二阶特性的考量——这些特性由海森矩阵所控制
 2. 当训练数据发生微小变化时，赫塞矩阵为快速重训练前馈网络提供了基础算法
 3. 海森矩阵的逆矩阵已被用于识别网络中权重最不重要的部分，作为网络"修剪"算法的组成部分



4. 在贝叶斯_{160/184}的拉普拉斯近似中，海森矩阵扮演核心角色
神经网络。其逆函数用于确定神经网络的预测分布。



海森矩阵：对角线近似

- 上述关于海森矩阵的部分应用需要的是海森矩阵的逆矩阵，而非海森矩阵本身。
- 因此，研究者开始关注对海森矩阵进行对角线近似的方法——
- 即仅将非对角线元素替换为零的对角近似，因其逆矩阵易于计算。
- 我们仍考虑误差函数 E_n
- 通过逐个模式分析，再对所有模式结果求和，即可获得海森矩阵。



海森矩阵：对角线近似

- 对于模式n，海森矩阵的对角元素可表示为

$$\frac{\partial^2 E_n}{\partial w_{ji}^2} = \frac{\partial^2 E_n}{\partial a_j^2} z_i^2$$

- 上述方程右侧的二阶导数可通过链式法则递归求得，从而得到形式为

$$\frac{\partial^2 E_n}{\partial a_j^2} = \sum_{k,k'} h_{pa_j}^2 q_{k,k'}^2 \frac{\partial^2 E_n}{\partial a_k \partial a_{k'}} = \sum_{k,k'} h_{pa_j}^2 q_{k,k'}^2 \frac{\partial E_n}{\partial a_k} \frac{\partial E_n}{\partial a_{k'}}$$

- 若现在忽略二阶导数项中的非对角线元素，则得到

$$\frac{\partial^2 E_n}{\partial a^2} \approx \sum_j h_{pa_j}^2 q_{jj}^2 \frac{\partial^2 E_n}{\partial a^2} = \sum_j h_{pa_j}^2 q_{jj}^2 \frac{\partial E_n}{\partial a} \frac{\partial E_n}{\partial a}$$



- 当神经网络应用于回归问题时，通常采用如下形式的平方和误差函数

$$E = \frac{1}{2} \sum_{n=1}^N (y_n - t_n)^2$$

- 为简化记号，此处仅考虑单输出情况（多输出情况的推广是直接的）。
- 因此可将海森矩阵表示为

$$H = \sum_{n=1}^N \nabla y_n \nabla y_n^T = \sum_{n=1}^N \nabla y_n \nabla y_n^T (y_n - t_n)^2 \quad (\text{外积})$$

- 若网络已在数据集上完成训练，且其输出 y_n 发生非常接近目标值，则第二项将很小



- 然而，更普遍地，基于以下论证忽略该项可能是合理的。
- 回顾使平方和损失最小化的最优函数即为目标数据的条件平均值。
- 此时量 $p_{y_n} t_n \mathbf{Q}$ 成为均值为零的随机变量。
- 若假设其值与（外积）右侧二阶导数项的值无关联，
- 那么在对 n 求和时，整个项的平均值将趋于零。



- 忽略（外积）中的第二项后，得到列文伯格-马夸特近似或外积近似，其表达式为

$$H \approx \sum_{n=1}^N b_n b_n^T$$

- 其中 $b_n = \nabla y_n - \nabla a_n$ 因为输出单元的激活函数仅为恒等函数。
- 评估过程相当简单，因为它仅涉及误差函数的一阶导数，可通过标准反向传播在 $O(pq)$ 步内高效计算。
- 矩阵的元素可通过简单操作在 $O(p^2)$ 中 q 步内求得



- 对于采用逻辑sigmoid输出单元激活函数的网络，其交叉熵误差函数对应的近似表达式为：
- 对应的近似表达式为

$$H = \sum_{n=1}^N y_n p_1 y_n q b_n b_n^T$$

- 对于具有softmax输出单元激活函数的多类网络，可以得到类似的结果。



- 我们可以利用外积近似法，建立一种计算效率高的程序来近似求解海森矩阵的逆矩阵。
- 首先将外积近似用矩阵形式表示为

$$H_N \approx \sum_{n=1}^N \mathbf{b}_n \mathbf{b}_n^T$$

- 其中 $\mathbf{b}_n = \nabla_{\mathbf{w}} a_n$ 是数据点 n 对输出单元激活函数梯度的贡献。



- 现推导一种逐次构建海森矩阵的序列化流程，通过逐个纳入数据点实现。
- 假设我们已利用前 L 个数据点获得了逆海森矩阵。
- 通过分离数据点 $L+1$ 的贡献项，可得：

$$H_{L+1} = H_L + b_{L+1} b_{L+1}^T$$



海森矩阵：逆海森矩阵

- 为了评估海森矩阵的逆矩阵，我们现在考虑单位矩阵：

$$M^{-1} = \frac{1}{v^T M^{-1} v} \left(M^{-1} v - \frac{v^T M^{-1} v}{v^T M^{-1} v} M^{-1} v \right) + \frac{v^T M^{-1} v}{v^T M^{-1} v} M^{-1} v$$

- 这只是伍德伯里恒等式的特例。
- 若将 H_L 视为 M ，并将 b_{L1} 视为 v ，则可得

$$H_{L1}^{-1} = H_L^{-1} - \frac{H_L^{-1} b_{L1} b_{L1}^T H_L^{-1}}{1 + b_{L1}^T H_L^{-1} b_{L1}}$$

- 通过这种方式，数据点被依次吸收，直到 $L = N$ ，整个数据集完成处理。



海森矩阵：逆海森矩阵

- 因此，该结果代表了一种通过单次遍历数据集来评估海森矩阵逆矩阵的过程。
- 初始矩阵 H_0 选取为 αI ，其中 α 为微小量，
- 使得算法实际求解的是 αI 的逆矩阵。
- 结果对 α 的具体取值并不特别敏感。
- 将该算法推广至具有多个输出的网络是直接可行的。

我们注意到，海森矩阵有时可作为网络训练算法的一部分间接计算。

特别是准牛顿非线性优化算法通过逐步构建



- 与误差函数的一阶导数类似，我们可通过有限差分法求得二阶导数，其精度受数值精度的限制。

- 若依次扰动所有可能的权重对，则得到

$$\frac{\partial^2 E}{\partial w_{ji} \partial w_{lk}} = \frac{1}{4\epsilon^2} \left(E(p_{w_{ji}+\epsilon, w_{lk}}) - E(p_{w_{ji}-\epsilon, w_{lk}}) - E(p_{w_{ji}, w_{lk}+\epsilon}) + E(p_{w_{ji}, w_{lk}-\epsilon}) \right) = O(\epsilon^2) \cdot q.$$

- 再次，通过使用对称中心差分公式，我们确保残差误差为 $O(\epsilon^2) \cdot q$ 而不是 $O(\epsilon q)$ 。



- 由于海森矩阵中包含 W^2 个元素，
- 由于每个元素的评估需要四次前向传播，每次传播都需要 $O_p W q$ 操作（按模式计算），
- 由此可见，该方法需执行 $O W^3$ 次运算才能求解完整的赫塞矩阵。
- 因此该方法具有较差的可扩展性，但在实践中作为反向传播算法软件实现的验证工具仍具有重要价值。



海森矩阵：有限差分

- 通过对误差函数的一阶导数应用中心差分，可获得更高效的数值微分版本—

- 这些导数本身通过反向传播计算获得。这得到

$$\frac{\partial^2 \mathcal{L}}{\partial w_{ji} \partial w_{lk}} = \frac{1}{2\epsilon} \left(\frac{\partial \mathcal{L}}{\partial w_{ji}} \right)^{pw} \frac{\partial \mathcal{L}}{\partial w_{lk}} + \mathcal{O}(\epsilon^2)$$

- 由于现在只需扰动w个权重，
- 且梯度可在 $\mathcal{O}(Wq)$ 步内计算，
- 由此可见该方法可在 $\mathcal{O}(W^2q)$ 次操作内求得海森矩阵。



- 对于任意前馈拓扑结构的网络，通过扩展用于计算一阶导数的反向传播技术，也能精确求解海森矩阵，该方法兼具反向传播的诸多优势特性，包括计算效率。
- 它可应用于任何可微误差函数，只要该函数能表示为网络输出的函数形式，并适用于具有任意可微激活函数的网络。
- 计算海森矩阵所需的步骤数呈 $O_p W^2 Q$ 级增长。



- 在此我们考虑具有两层权重的网络特例，其所需方程可轻松推导。
- 我们将使用索引 i 和 i^1 表示输入，索引 j 和 j^1 表示隐藏单元，索引 k 和 k^1 表示输出。
- 首先定义

$$\delta_k = \frac{\partial E_n}{\partial a_k} \quad M_{kk'} = \frac{\partial^2 E_n}{\partial a_k \partial a_{k'}}$$

- 其中 E_n 表示数据点 n 对误差的贡献。



- 该网络的赫塞矩阵可分为三个独立块进行分析：

- 第二层的两个权重：

$$\frac{B^2 E_n}{Bw_{kj}^{p2q} Bw_{k'j'}^{p2q}} \cdot z_j z_{j'} M_{kk'}$$

- 第一层的两个权重：

$$\frac{B^2 E_n}{Bw_{ji}^{p1q} Bw_{j'i'}^{p1q}} \cdot x_i x_{i'} h^2 p a_{j'q} I_{jj'} w_k \cdot \delta_k \cdot x_{i'} x_{i'} h^1 p a_{j'q} h^1 p a_{j'q} w_{k,k'} \cdot M_{kk'}$$

- 每层一个权重：

$$\frac{B^2 E_n}{p1q \quad p2q} \cdot x_i h^1 p a_{j'q} \cdot \delta_k h^1 p a_{j'q} \cdot z_j \cdot Bw_{kj'}^{p2q} M_{kk'}$$



- 对于许多数克莱因矩阵的应用，真正关注的并非矩阵 H 本身
- 而是 H 与某个向量 v 的乘积。
- 海森矩阵的计算需要 $O(pW^2 Q)$ 次操作，同时还需占用 $O(pW^2 Q)$ 的存储空间。
- 然而，我们需要计算的向量 $v^T H$ 仅有 W 个元素，
- 因此无需将海森矩阵作为中间步骤计算，可直接尝试
尝试寻找一种高效方法，直接计算 $v^T H$ 且仅需 $O(pWq)$ 次运算。



- 为此，首先注意到：

$$\mathbf{v}^T \mathbf{H} \mathbf{H} \mathbf{v}^T \nabla p \nabla E q$$

- 我们可据此推导出用于计算 ∇E 的标准前向传播与反向传播方程

- 并将上述方程应用于这些方程，从而得到一组

用于评估 $\mathbf{v}(\tau)$ 的正向传播和反向传播方程组

$$\mathbf{v}^T \mathbf{H}_0$$

- 这相当于对原始前向传播和反向传播方程施加微分算子 $\mathbf{v}^T \nabla$ 。

- Pearlmutter (1994) 采用符号 $\nabla_{\mathbf{v}}$ 表示算子 $\mathbf{v}^T \nabla$ 。



- 分析过程相当直接，运用了常微分计算的常规规则，并结合了以下结果

$$Yw = v_o$$

- 该技术通过简单示例最能说明其原理，我们再次选择具有线性输出单元和平方和误差函数的两层网络。
- 我们考虑数据集中某一模式对误差函数的贡献。
- 所需向量随后通过常规方式获得，即分别对每个模式的贡献求和。
各模式的贡献值进行求和。



- 对于双层网络，前向传播方程由以下公式给出：

$$\begin{aligned} a_j &= \sum_i w_{ji} x_i \\ z_j &= h p_a a_j \\ y_k &= \sum_j w_{kj} z_j \end{aligned}$$

- 对这些方程应用 ∇ 运算符，得到

$$\begin{aligned} \nabla a_j &= \sum_i v_{ji} \nabla x_i \\ \nabla z_j &= h^1 p_a \nabla a_j \\ \nabla y_k &= \sum_j w_{kj} \nabla z_j \end{aligned}$$

- 其中 v_{ji} 是向量 v 中与权重 w_{ji} 对应的元素。



- 形式为 Y_{z_j} 、 Y_{a_j} 和 Y_{y_k} 的量应视为新变量，其值通过上述方程求得。
- 由于我们考虑的是均方误差函数，因此有以下标准反向传播表达式：

$$\delta_k = y_k - t_k$$

$$\delta_j = h^1_{pa_j} \mathbf{Q} \sum_k w_{kj} \delta_k$$

- 再次，我们对这些方程应用 ∇ 算子，得到一组形式为

$$\nabla Y t \delta_k \mathbf{u} = \nabla Y t y_k \mathbf{u}$$

$$\nabla Y t \delta u_j = h^2_{pa_j} \mathbf{Q} \nabla Y t a u_j = \sum_k w_{kj} \delta_k = h^1_{pa_j} \mathbf{Q} \sum_k v_{kj} \delta_k = h^1_{pa_j} \mathbf{Q} \sum_k w_{kj} \nabla Y t \delta_k$$



- 最后，我们得到误差一阶导数的常规方程：

$$B_{wkj} \frac{\partial E}{\partial z_{kj}} \delta z_{kj} + B_{wji} \frac{\partial E}{\partial x_{ji}} \delta x_{ji}$$

- 并用 Y 的运算符作用于这些元素，我们得到向量 v^T 的元素表达式：

$$Y \frac{\partial E}{\partial z_{kj}} \delta z_{kj} + Y \frac{\partial E}{\partial x_{ji}} \delta x_{ji} \quad (乘法)$$



- 该算法的实现涉及引入额外变量
 - $Y_{ta_j} u, Y_{tz_j} u$ 以及 $Y_{t\delta_j} u$ 用于隐藏层单元
 - 以及输出单元的 $Y_{t\delta_k} u$ 和 $Y_{ty_k} u$ 。
- 对于每个输入模式，可通过上述结果求得这些量的值
- $v^T H$ 的元素由（乘法）给出。
- 该方法的优雅之处在于：计算 $v^T H$ 的方程与标准的前向传播和反向传播过程高度相似，
- 因此，现有软件扩展以计算此乘积通常



若需计算全海森矩阵，可通过选择向量 v 依次取如下形式的单位向量序列实现：

$$p_0, 0, \dots, 1, \dots, 0_q$$

每个单位向量分别对应海森矩阵的一列。